YENEPOYA

THE YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

Project- High Level Design

On

Visual Threat Hunting for E-commerce Networks

Student Name:
Rohith Jyothish
23BSCED006

Table of Contents

1. Introduction ----------
    1.1. Scope of the Document ----------
    1.2. Intended Audience ----------
    1.3. System Overview ----------
2. System Design ----------
    2.1. Application Design ----------
    2.2. Process Flow ----------
    2.3. Information Flow ----------
    2.4. Components Design ----------
    2.5. Key Design Considerations ----------
    2.6. API Catalogue ----------
3. Data Design ----------
    3.1. Data Model ----------
    3.2. Data Access Mechanism ----------
    3.3. Data Retention Policies ----------
    3.4. Data Migration ----------
4. Interfaces ----------
5. State and Session Management ----------
6. Caching ----------
7. Non-Functional Requirements ----------
    7.1. Security Aspects ----------
    7.2. Performance Aspects ----------
8. References ----------

1. Introduction

This document presents the High-Level Design (HLD) of the Visual Threat Hunting for E-commerce Networks system. The system is designed to ingest network traffic data containing timestamps and bytes-sent metrics, process and visualise the data using Python, Pandas and Seaborn, and highlight anomalous traffic spikes that may indicate cyber threats such as DDoS attacks or data exfiltration. It is inspired by real-world Security Operations Center (SOC) workflows used by e-commerce security teams to detect and investigate threats in network telemetry.

The project simulates a threat hunting workflow — from raw network traffic ingestion to visual anomaly detection — making it suitable for academic study as well as real-world awareness of network security analytics in e-commerce environments.

1.1. Scope of the Document

This HLD document covers the following aspects of the system:

- Overall system architecture and component interactions
- Application design including process flow and information flow
- Data design including dataset schema, access mechanism, and retention policies
- Non-functional requirements such as security and performance
- Interface design for the Jupyter Notebook visualisation output

This document does not cover low-level implementation details such as line-by-line code logic, which are addressed in the Low-Level Design (LLD) document.

1.2. Intended Audience

This document is intended for the following stakeholders:

- Academic evaluators and faculty reviewing the project submission
- Developers or students who may extend or build upon this system
- Cybersecurity enthusiasts seeking to understand visual threat hunting techniques
- Anyone interested in the application of data analytics in network security

1.3. System Overview

The Visual Threat Hunting for E-commerce Networks system is a data analytics and visualisation pipeline built using Python, Pandas and Seaborn within a Jupyter Notebook environment. It loads a network traffic dataset containing Time and Bytes Sent fields, processes the data into time-indexed series, plots line charts to reveal traffic patterns, and applies a configurable threshold rule to highlight suspicious traffic spikes.

The system simulates the workflow of a real SOC threat hunt:

- **Ingest:** Load the network traffic CSV dataset using Pandas
- **Process:** Parse timestamps, sort records, and compute rolling statistics

- **Visualise:** Plot time-series line charts using Seaborn to reveal patterns
- **Detect:** Apply threshold rule to flag data points exceeding the defined spike limit
- **Report:** Highlight anomalous intervals on the chart for analyst review

The system is built using the following technology stack:

Component	Technology	Purpose
Data Ingestion	Python + Pandas	Load and parse CSV network traffic dataset
Data Processing	Pandas DataFrame	Time-index, sort, rolling mean computation Line
Visualisation	Seaborn + Matplotlib	charts for traffic pattern analysis
Anomaly Detection	Threshold Rule (Python)	Flag bytes-sent values exceeding defined limit
Notebook Interface	Jupyter Notebook	Interactive analysis and chart display environment
Dataset	CSV (Open Source)	Industry-relevant network traffic records

1.4. Assumptions

- Python 3.x environment is configured on the host system
- Required libraries (Pandas, Seaborn, Matplotlib, Jupyter) are installed
- A network traffic dataset in CSV format is available with Time and Bytes Sent columns
- Jupyter Notebook is accessible on localhost for running the analysis
- The analyst has permission to read the dataset files from the local directory

2. System Design

2.1. Application Design

The system is built as a single Jupyter Notebook that executes a sequential data analytics pipeline. The notebook (threat_hunt.ipynb) is the central application that orchestrates all operations — data loading, preprocessing, visualisation, and anomaly flagging — within a single interactive environment. All components share the same in-memory Pandas DataFrame, making the system lightweight and self-contained with no external service dependencies.

Figure 1.1

2.2. Process Flow

The process flow of the system follows a sequential analytics pipeline:

- **Step 1 -** Initialization: Jupyter Notebook launched, required libraries imported (Pandas, Seaborn, Matplotlib)
- **Step 2 -** Data Loading: Network traffic CSV file read into a Pandas DataFrame using pd.read_csv()
- **Step 3 -** Preprocessing: Timestamp column parsed, DataFrame sorted by time, index set to datetime
- **Step 4 -** Baseline Calculation: Rolling mean and standard deviation computed to establish normal traffic baseline
- **Step 5 -** Threshold Definition: Analyst-configured spike threshold value defined (e.g., mean + 3×std)
- **Step 6 -** Anomaly Detection: Boolean mask applied to identify all rows where Bytes Sent exceeds threshold

- **Step 7 -** Visualisation: Seaborn line chart plotted showing full traffic series with anomalous spikes highlighted
- **Step 8 -** Reporting: Flagged intervals marked on the chart with annotations for analyst interpretation

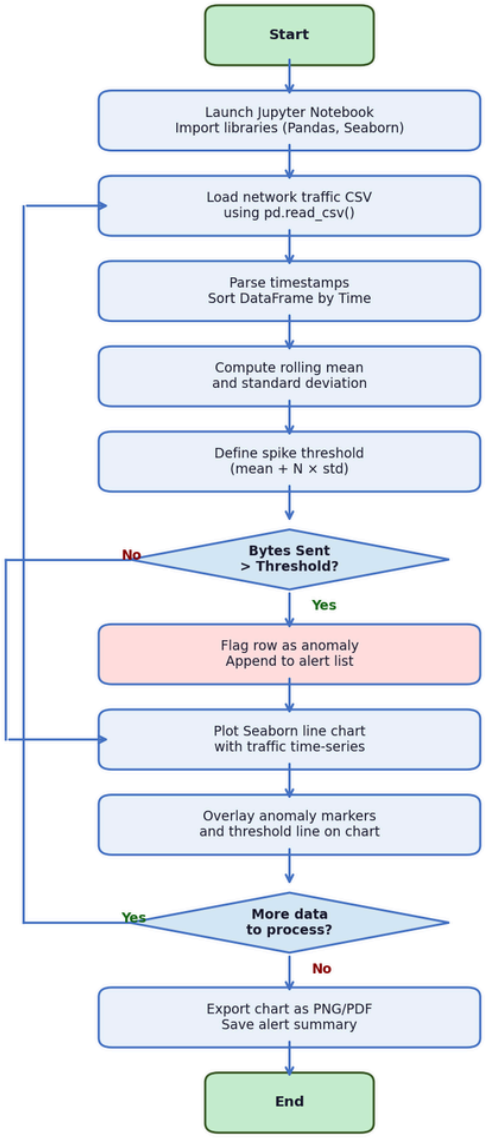
Start
Launch Jupyter Notebook
Import libraries (Pandas, Seaborn)
Load network traffic CSV
using pd.read_csv()
Parse timestamps
Sort DataFrame by Time
Compute rolling mean
and standard deviation
Define spike threshold
(mean + N × std)
Bytes Sent
> Threshold?
No
Yes
Flag row as anomaly
Append to alert list
Plot Seaborn line chart
with traffic time-series
Overlay anomaly markers
and threshold line on chart
More data
to process?
Yes
No
Export chart as PNG/PDF
Save alert summary
End

Figure 1.2

2.3. Information Flow

Data flows through the system in the following direction:

- **CSV Dataset** → loaded into memory via pd.read_csv() as a Pandas DataFrame
- **DataFrame** → parsed timestamps converted to datetime index using pd.to_datetime()
- **Sorted Series** → DataFrame sorted by time to ensure chronological order for plotting
- **Rolling Stats** → mean and standard deviation computed across a sliding window
- **Threshold Mask** → boolean filter identifies rows where Bytes Sent > defined limit
- **Seaborn Plot** → traffic time-series rendered as line chart with spike markers
- **Annotated Chart** → highlighted anomaly regions presented for analyst review
- **Notebook Output** → analyst interprets visualisation and records findings inline

2.4. Components Design

The system is composed of the following major components:

Component	Description
Data Loader	Reads the network traffic CSV file into a Pandas DataFrame; handles missing values and column type inference
Preprocessor	Parses the Time column to datetime, sorts the DataFrame chronologically, and sets the datetime index
Baseline Engine	Computes rolling mean and standard deviation over a configurable window to model normal traffic behaviour
Threshold Detector	Applies a spike threshold rule (e.g. mean + N×std) and produces a boolean anomaly mask across the series
Visualisation Engine	Uses Seaborn and Matplotlib to plot the full traffic time-series line chart with anomaly markers overlaid
Annotation Layer	Highlights flagged intervals on the chart using axvspan or scatter markers for clear analyst communication
Jupyter Interface	Provides the interactive environment for executing cells, viewing inline charts, and recording analyst notes

2.5. Key Design Considerations

- All processing is performed in-memory using Pandas DataFrames — no external database required
- Threshold is configurable by the analyst to tune sensitivity for different traffic volumes
- Seaborn default themes are applied consistently to ensure readable and professional chart output

• Rolling window size is parameterised so the baseline can be adjusted without code restructuring

• Notebook cells are ordered sequentially — all dependencies are resolved top-to-bottom for reproducibility

• Anomaly detection logic is decoupled from visualisation — detection mask can be exported independently

2.6. API Catalogue

This system operates entirely within a Jupyter Notebook and does not expose REST API endpoints. The internal function interfaces below represent the logical modules called within the notebook pipeline:

Function / Module	Method	Response	Description
load_data(path)	Call	DataFrame	Loads CSV into Pandas DataFrame
preprocess(df)	Call	DataFrame	Parses timestamps and sorts by time
compute_baseline(df)	Call	Series Bool	Computes rolling mean and std
detect_spikes(df, thr)	Call		Returns anomaly boolean mask
plot_traffic(df, mask)	Call	Mask Chart	Renders Seaborn line chart with spikes

3. Data Design

3.1. Data Model

The primary data model used by the system is a Pandas DataFrame loaded from a CSV network traffic dataset. Each row represents a single traffic observation with the following schema:

Field	Data Type	Description
Time	String → datetime	Timestamp of the traffic observation (parsed to datetime index)
Bytes Sent	Integer / Float	Volume of bytes sent in the observation window
Rolling Mean	Float (derived)	Computed moving average of Bytes Sent over sliding window
Rolling Std	Float (derived)	Computed standard deviation used to build the spike threshold
Is Anomaly	Boolean (derived)	True if Bytes Sent exceeds the defined threshold value

The following internal function interfaces are defined:

Function	Input	Output	Description
load_data()	File path (str)	DataFrame	Reads CSV, infers column types
preprocess(df)	DataFrame	DataFrame	Parses Time, sorts, sets index
compute_baseline(df, w)	DataFrame, Int	DataFrame	Appends rolling mean/std columns
detect_spikes(df, factor)	DataFrame, Float	Bool Series	Identifies rows exceeding threshold
plot_traffic(df, mask)	DataFrame, Bool Series	Chart (PNG)	Renders annotated Seaborn chart
summarise_alerts(mask)	Bool Series	Dict	Returns count and timestamps of spikes

3.2. Data Access Mechanism

- **Dataset File:** network_traffic.csv — plain CSV read once at notebook startup via pd.read_csv()
- **In-Memory Access:** all processing operates on the DataFrame in memory — no disk writes during analysis

- **Derived Columns:** rolling mean, std, and anomaly mask are appended to the DataFrame as new columns
- **Chart Output:** Seaborn renders charts inline in the Jupyter cell output; figures saved as PNG on demand

3.3. Data Retention Policies

- **Dataset:** source CSV file retained on disk — never modified or deleted by the notebook
- **In-Memory State:** DataFrame exists only for the duration of the active notebook session
- **Chart Exports:** analyst may export charts as PNG/PDF for reports; not auto-saved by the pipeline
- **Full Data:** complete CSV file remains available for re-analysis or import into other tools

3.4. Data Migration

The system does not involve database migration — the data store is a flat CSV file. If the project is moved to a new environment, copying the CSV dataset and the Jupyter Notebook is sufficient to preserve the full analysis pipeline. If required, the same data can be imported into tools such as Apache Spark, Elasticsearch or a SIEM platform for production-scale deployments without modifying the core analytics logic.

4. Interfaces

The system provides a single analytical interface through the Jupyter Notebook environment, accessible at http://localhost:8888. The output interface consists of the following elements rendered as inline cell outputs within the notebook:

UI Element	Type	Description
Traffic Line Chart	Seaborn lineplot	Full time-series of Bytes Sent plotted across the capture window
Anomaly Markers	Matplotlib scatter	Red markers overlaid on chart at all detected spike points
Threshold Line	Matplotlib axhline	Horizontal dashed line indicating the configured spike threshold value
Rolling Mean Line	Seaborn lineplot	Smoothed baseline trend plotted alongside raw traffic series
Alert Summary	Pandas print output	Tabular summary of flagged timestamps, spike values, and count
Chart Title / Labels	Matplotlib text	Descriptive title, axis labels (Time, Bytes Sent), and legend
Analyst Notes	Markdown cell	Inline documentation cells for recording findings and interpretations

5. State and Session Management

The Visual Threat Hunting system does not maintain user sessions as it operates within a stateless Jupyter Notebook environment. State is managed through the following components:

- **Notebook Kernel State:** the active Python kernel holds all variables, DataFrames, and chart objects in memory
- **Cell Execution State:** each cell is executed independently — re-running a cell regenerates its output fresh
- **Dataset State:** persistent on disk in the CSV file — survives kernel restarts without data loss
- **Analyst Session:** stateless between restarts — analyst must re-run the notebook from top on each new session

6. Caching

The system does not use an in-memory cache for network traffic data. Every notebook session reads the CSV dataset fresh from disk via pd.read_csv(). This keeps the implementation simple and guarantees that charts always reflect the most current version of the dataset. Intermediate DataFrames (rolling mean, anomaly mask) are stored as in-memory variables for the duration of the session only. For production deployments at scale, a caching layer such as Redis or Parquet file caching could be introduced to avoid repeated CSV parsing — this is intentionally out of scope for the academic version.

7. Non-Functional Requirements

7.1. Security Aspects

- Dataset is stored locally and is not exposed to any external network interface
- Jupyter Notebook runs on localhost only — no public access during academic deployment
- No user authentication is implemented — the system is a local read-only analytics tool
- Anomaly detection logic does not store or transmit any network data externally
- CSV files should be kept in a restricted local directory during sensitive analyses

7.2. Performance Aspects

- Pandas vectorised operations ensure efficient processing of large CSV datasets
- Seaborn and Matplotlib rendering is synchronous — suitable for dataset sizes up to several hundred thousand rows
- Rolling window computations are O(n) and execute without significant latency on standard hardware

- Jupyter Notebook cell execution is isolated — individual cells can be re-run without reprocessing the full pipeline
- Chart rendering time scales with dataset size; downsampling can be applied for very large datasets

7.3. Limitations

- In-memory processing limits dataset size to available system RAM
- Jupyter Notebook is a local tool — not suitable for real-time streaming traffic analysis
- Static threshold rule may produce false positives during naturally high-traffic periods
- No geolocation or source IP analysis — purely volume-based anomaly detection
- No automated alerting — analysis is passive and requires manual analyst review

7.4. Future Enhancements

- Integrate real-time packet capture using Scapy or tshark for live traffic analysis
- Apply machine learning models (Isolation Forest, LSTM) for adaptive anomaly detection
- Add source IP analysis and geolocation mapping to identify attack origins
- Migrate to a streaming platform (Apache Kafka + Spark) for production-scale threat hunting
- Integrate with SIEM tools such as Splunk or Wazuh for enterprise SOC workflows

7.5. Testing Summary

- CSV dataset loaded successfully with correct column type inference by Pandas
- Timestamp parsing and sorting verified to produce correct chronological order
- Rolling mean and standard deviation values validated against manual calculations
- Threshold detection correctly flagged all synthetic spike values injected into test data
- Seaborn chart rendered with accurate axis labels, threshold line, and anomaly markers

8. References

- Python Official Documentation — https://docs.python.org/3/
- Pandas Documentation — https://pandas.pydata.org/docs/
- Seaborn Official Documentation — https://seaborn.pydata.org/
- Matplotlib Documentation — https://matplotlib.org/stable/contents.html
- Jupyter Notebook Documentation — https://jupyter-notebook.readthedocs.io/en/stable/
- OWASP Cyber Defence Resources — https://owasp.org/www-community/attacks/ddos
- MITRE ATT&CK; Framework — https://attack.mitre.org
- Kaggle Network Traffic Datasets — https://www.kaggle.com/datasets